



Lab Activity



Week 10: 2D Lists and Matrix Operations - Grids and Tables



Duration: 60 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Create 2D lists (lists of lists) correctly
- Avoid the common `[[0]*n]*m` aliasing trap
- Access elements using two indices `grid[row][col]`
- Traverse 2D lists with nested loops
- Build grid-based programs (tic-tac-toe, matrices)

Getting Started

- Extract the given [week10_lab.zip](#) file to your desktop
- **KEY CONCEPT:** 2D lists are lists that contain other lists!
- Always use `[[val for j in range(cols)] for i in range(rows)]`

1 Exercise 1: Understanding 2D Lists (8 minutes)

Learn what 2D lists are and how to create them!

Open [week10_ex1.py](#) and complete:

```
1  # Create a 2D list manually
2  grid = [[1, 2, 3],
3          [4, 5, 6],
4          [7, 8, 9]]
5
6  # Access elements using two indices
7  print("Element at row 0, col 0:", grid[___][___]) # Should print 1
8  print("Element at row 1, col 2:", grid[___][___]) # Should print 6
9  print("Element at row 2, col 1:", grid[___][___]) # Should print 8
10
11 # Access entire rows
12 print("\nFirst row:", grid[___])
13 print("Last row:", grid[___]) # Use negative index
14
15 # Modify an element
16 grid[1][1] = ___ # Change center to 0
17 print("\nAfter modification:", grid)
18
19 # Get dimensions
```

```

20 rows = len(grid)
21 cols = len(grid[0]) # Length of first row
22 print(f"\nGrid is {rows}x{cols}")

```

💡 **Tip:** Think "row first, column second" - grid[row][col]!

✅ **Checkpoint:** Can you access 2D list elements? Show teacher!

2 Exercise 2: The Wrong Way vs Right Way (10 minutes)

Learn to avoid the dangerous aliasing trap!

Open [week10_ex2.py](#) and complete:

```

1  # WRONG WAY - Creates aliased rows!
2  print("=== WRONG WAY ===")
3  rows, cols = 3, 4
4  bad_grid = [[0] * cols] * rows # BUG!
5
6  print("Initial grid:")
7  for row in bad_grid:
8      print(row)
9
10 # Try to modify one element
11 bad_grid[0][0] = 1
12
13 print("\nAfter changing [0][0] to 1:")
14 for row in bad_grid:
15     print(row) # ALL first elements changed!
16
17 print("\nBUG: All rows are the SAME object!")
18
19 # RIGHT WAY - List comprehension
20 print("\n" + "=" * 40)
21 print("=== RIGHT WAY ===")
22 rows, cols = 3, 4
23 good_grid = [[0 for j in range(cols)] for i in range(rows)]
24
25 print("Initial grid:")
26 for row in good_grid:
27     print(row)
28
29 # Modify one element
30 good_grid[0][0] = 1 # Fill in: 1
31
32 print("\nAfter changing [0][0] to 1:")
33 for row in good_grid:
34     print(row) # Only first row changed!
35
36 print("\nEach row is an independent object!")
37
38 # Alternative: Using loops
39 grid2 = []
40 for i in range(rows):
41     row = [] # Create NEW list each time
42     for j in range(cols):

```

```

43         row.append(0)
44         grid2.append(row)
45
46 print("\nGrid created with loops:")
47 for row in grid2:
48     print(row)

```

 **Memory Diagram:** Use list comprehension to ensure each row is independent!

 **Pair Programming:** Switch who types every 5 minutes!

3 Exercise 3: Traversing 2D Lists (10 minutes)

Master nested loops for processing grids!

Open [week10_ex3.py](#) and complete:

```

1  # Create a number grid
2  matrix = [[1, 2, 3],
3            [4, 5, 6],
4            [7, 8, 9]]
5
6  # Pattern 1: Print all elements
7  print("=== ALL ELEMENTS ===")
8  for row in range(len(matrix)):
9      for col in range(len(matrix[row])):
10         print(f"[{row}][{col}] = {matrix[row][col]}")
11
12 # Pattern 2: Print row by row
13 print("\n=== ROW BY ROW ===")
14 for row in matrix:
15     for element in row:
16         print(element, end=" ")
17     print() # New line after each row
18
19 # Pattern 3: Calculate sum of all elements
20 total = 0
21 for row in matrix:
22     for element in row:
23         total += element
24 print(f"\nSum of all elements: {total}")
25
26 # Pattern 4: Find maximum
27 max_val = matrix[0][0]
28 for row in matrix:
29     for element in row:
30         if element > max_val:
31             max_val = element
32 print(f"Maximum element: {max_val}")
33
34 # Practice: Count even numbers
35 even_count = 0
36 for row in matrix:
37     for element in row:
38         if -----: # Check if even
39             even_count += 1
40 print(f"Even numbers: {even_count}")

```

✓ **Checkpoint:** Can you traverse 2D lists? Excellent!

4 Exercise 4: Building a Tic-Tac-Toe Board (12 minutes)

Create a real game board!

Open `week10_ex4.py` and complete:

```

1  # Initialize empty board
2  board = [[' ' for j in range(3)] for i in range(3)]
3
4  def display_board(board):
5      """Display the board nicely"""
6      print("\n  0   1   2")
7      for i in range(len(board)):
8          print(i, end=" ")
9          for j in range(len(board[i])):
10             print(board[i][j], end=" ")
11             if j < 2:
12                 print(" | ", end=" ")
13             print()
14             if i < 2:
15                 print(" " + "-" * 11)
16         print()
17
18 # Display empty board
19 display_board(board)
20
21 # Place some moves
22 board[1][1] = 'X' # Center
23 board[0][0] = 'O' # Top-left
24 board[2][2] = 'X' # Bottom-right
25
26 display_board(board)
27
28 # Check if a position is empty
29 def is_empty(board, row, col):
30     """Returns True if position is empty"""
31     return board[row][col] == ___ # Fill in: ' '
32
33 # Check for winner in a row
34 def check_row(board, row, player):
35     """Check if player has won in given row"""
36     for col in range(3):
37         if board[row][col] != player:
38             return False
39     return True
40
41 # Test winner check
42 board[0] = ['X', 'X', 'X'] # X wins!
43 display_board(board)
44
45 if check_row(board, 0, 'X'):
46     print("X wins on row 0!")
47
48 # Practice: Complete these functions
49 def check_column(board, col, player):

```

```

50     """Check if player has won in given column"""
51     for row in range(3):
52         if board[___][___] != player:
53             return False
54     return True
55
56 def check_diagonal(board, player):
57     """Check main diagonal (top-left to bottom-right)"""
58     for i in range(3):
59         if board[i][i] != player:
60             return False
61     return True

```

✔ Checkpoint: Tic-tac-toe board working? Awesome!

5 Exercise 5: Matrix Operations (10 minutes)

Work with mathematical matrices!

Open [week10_ex5.py](#) and complete:

```

1  # Create two matrices
2  matrix_a = [[1, 2, 3],
3              [4, 5, 6]]
4
5  matrix_b = [[7, 8, 9],
6              [10, 11, 12]]
7
8  # Add two matrices element by element
9  def add_matrices(a, b):
10     """Add two matrices of same size"""
11     rows = len(a)
12     cols = len(a[0])
13
14     result = [[0 for j in range(cols)] for i in range(rows)]
15
16     for i in range(rows):
17         for j in range(cols):
18             result[i][j] = a[i][j] + b[i][j]
19
20     return result
21
22 sum_matrix = add_matrices(matrix_a, matrix_b)
23 print("Matrix A:")
24 for row in matrix_a:
25     print(row)
26
27 print("\nMatrix B:")
28 for row in matrix_b:
29     print(row)
30
31 print("\nA + B:")
32 for row in sum_matrix:
33     print(row)
34
35 # Transpose a matrix (swap rows and columns)

```

```

36 def transpose(matrix):
37     """Return transposed matrix"""
38     rows = len(matrix)
39     cols = len(matrix[0])
40
41     result = [[0 for j in range(rows)] for i in range(cols)]
42
43     for i in range(rows):
44         for j in range(cols):
45             result[___][___] = matrix[i][j] # Fill in: j, i
46
47     return result
48
49 original = [[1, 2, 3],
50             [4, 5, 6]]
51
52 transposed = transpose(original)
53 print("\n\nOriginal matrix:")
54 for row in original:
55     print(row)
56
57 print("\n\nTransposed:")
58 for row in transposed:
59     print(row)
60
61 # Practice: Multiply each element by a scalar
62 def scalar_multiply(matrix, scalar):
63     """Multiply every element by scalar"""
64     rows = len(matrix)
65     cols = len(matrix[0])
66     result = [[0 for j in range(cols)] for i in range(rows)]
67
68     for i in range(rows):
69         for j in range(cols):
70             result[i][j] = matrix[i][j] * _____ # Fill in: scalar
71
72     return result
73
74 doubled = scalar_multiply(original, 2)
75 print("\n\nOriginal * 2:")
76 for row in doubled:
77     print(row)

```

6 Exercise 6: Grid-Based Challenge (10 minutes)

Put it all together in a complete program!

Open [week10_ex6.py](#) and complete:

```

1 # Create a seating chart
2 print("=== CLASSROOM SEATING CHART ===")
3
4 # Create 4 rows, 5 seats per row
5 rows, cols = 4, 5
6 classroom = [[' ' for j in range(cols)] for i in range(rows)]
7
8 # Add some students (initials)
9 classroom[0][0] = 'AS' # Ali

```

```

10 classroom[0][2] = 'SF' # Sara
11 classroom[1][1] = 'AH' # Ahmed
12 classroom[2][3] = 'FM' # Fatima
13
14 # Display seating chart
15 def display_classroom(room):
16     """Show classroom layout"""
17     print("\n", end="")
18     for col in range(len(room[0])):
19         print(f" {col} ", end="")
20     print("\n" + " " + "-" * (len(room[0]) * 5 + 3))
21
22     for i in range(len(room)):
23         print(f"{i} | ", end="")
24         for j in range(len(room[i])):
25             seat = room[i][j] if room[i][j] != ' ' else '--'
26             print(f" {seat} ", end="")
27         print()
28     print()
29
30 display_classroom(classroom)
31
32 # Count empty seats
33 def count_empty(room):
34     """Count how many seats are empty"""
35     count = 0
36     for row in room:
37         for seat in row:
38             if seat == ' ': # Fill in: ' '
39                 count += 1
40     return count
41
42 empty = count_empty(classroom)
43 total = rows * cols
44 print(f"Empty seats: {empty}/{total}")
45
46 # Find a student
47 def find_student(room, initials):
48     """Find student's position, return (row, col) or None"""
49     for i in range(len(room)):
50         for j in range(len(room[i])):
51             if room[i][j] == initials:
52                 return (i, j)
53     return None
54
55 position = find_student(classroom, 'SF')
56 if position:
57     row, col = position
58     print(f"Sara (SF) sits at row {row}, seat {col}")
59
60 # Challenge: Move a student
61 def move_student(room, old_row, old_col, new_row, new_col):
62     """Move student from old position to new position"""
63     if room[new_row][new_col] == ' ': # Check if empty
64         student = room[old_row][old_col]
65         room[new_row][new_col] = student
66         room[old_row][old_col] = ' '
67         return True
68     return False
69
70 # Try to move Ali

```

```
71 if move_student(classroom, 0, 0, 1, 0):
72     print("\nMoved Ali to new seat!")
73     display_classroom(classroom)
```

✔ **Checkpoint:** Complete classroom program? Outstanding!

7 Lab Summary

✔ **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Created 2D lists with list comprehension
- ☐ Avoided the `[[0]*n]*m` aliasing trap
- ☐ Accessed elements using `grid[row][col]`
- ☐ Traversed 2D lists with nested loops
- ☐ Built a tic-tac-toe board
- ☐ Performed matrix operations
- ☐ Created a practical grid application
- ☐ Drew memory diagrams to understand structure

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	☹ Need Help	😬 Getting It	😊 Got It!
2D list structure	☐	☐	☐
Creating 2D lists correctly	☐	☐	☐
Traversing with nested loops	☐	☐	☐
Grid-based applications	☐	☐	☐

Why is `[[0]*n]*m` dangerous?

What grid-based program would you like to build?

😊 **Fantastic! You've mastered 2D lists!**